

Write a program to create a new process that exec a new program using system calls fork(), execlp() & wait()

fork()

The fork() is one of the syscalls that is very special and useful in Linux/Unix systems. It is used by processes to create the processes that are copies of themselves. With the help of such system calls, the child process can be created by the parent process. Until the child process is executed completely, the parent process is suspended.

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>

int main(int argc, char **argv)
{
    pid_t pid;
    pid = fork();
    if(pid==0)
    {
        printf("It is the child process and pid is %d\n",getpid());
        exit(0);
    }
    else if(pid > 0)
    {
        printf("It is the parent process and pid is %d\n",getpid());
    }
    else
    {
        printf("Error while forking\n");
        exit(EXIT_FAILURE);
    }
    return 0;
}
```

Output:

exec()

The exec() is such a system call that runs by replacing the current process image with the new process image. However, the original process remains as a new process but the new process replaces the head data, stack data, etc. It runs the program from the entry point by loading the program into the current process space.

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <stdlib.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
main(void) {
```

```
    pid_t pid = 0;
```

```
    int status;
```

```
    pid = fork();
```

```
    if (pid == 0) {
```

```
        printf("I am the child.");
```

```
execl("/bin/ls", "ls", "-l", "/home/ubuntu/", (char *) 0);

perror("In exec(): ");

}
```

```
if (pid > 0) {

    printf("I am the parent, and the child is %d.\n", pid);

    pid = wait(&status);

    printf("End of process %d: ", pid);
```

```
    if (WIFEXITED(status)) {

        printf("The process ended with exit(%d).\n", WEXITSTATUS(status));

    }
```

```
    if (WIFSIGNALED(status)) {

        printf("The process ended with kill -%d.\n", WTERMSIG(status));

    }

}
```

```
if (pid < 0) {
```

```
perror("In fork():");  
  
}
```

```
exit(0);  
  
}
```

OUTPUT:

wait()

As in the case of a fork, child processes are created and get executed but the parent process is suspended until the child process executes. In this case, a wait() system call is activated automatically due to the suspension of the parent process. After the child process ends the execution, the parent process gains control again.

```
#include<stdio.h> // printf()
```

```
#include<stdlib.h> // exit()
```

```
#include<sys/types.h> // pid_t
```

```
#include<sys/wait.h> // wait()
```

```
#include<unistd.h> // fork
```

```
int main(int argc, char **argv)
```

```
{
```

```
pid_t pid;
```

```
pid = fork();
```

```
if(pid==0)
```

```
{
```

```
printf("It is the child process and pid is %d\n",getpid());
```

```
int i=0;
```

```
for(i=0;i<8;i++)
```

```
{
```

```
printf("%d\n",i);
```

```
}
```

```
exit(0);
```

```
}
```

```
else if(pid > 0)
```

```
{
```

```
printf("It is the parent process and pid is %d\n",getpid());
```

```
int status;
```

```
wait(&status);
```

```
printf("Child is reaped\n");
```

```
}
```

```
else
```

```
{
```

```
printf("Error in forking..\n");
```

```
exit(EXIT_FAILURE);
```

```
}
```

```
return 0;
```

```
}
```

Output: